# 科学智能中的函数调用：实验自动化与数据集设计的综合性综述

## 摘要

本报告全面综述了科学智能（AI4Science）领域中实验自动化与函数调用的现状，重点关注生物、化学和药物研发领域。报告详细阐述了常见实验室仪器的可编程接口、学术界“自驱动实验室”的架构范式，以及工业界自动化平台的集成模式。通过分析现有函数调用数据集和大型语言模型（LLM）与自动化系统的集成案例，报告揭示了当前的技术能力与挑战。最终，报告提出了构建高质量科学智能函数调用数据集的战略性建议，强调了模式设计、元数据丰富以及对安全性、延迟和异步执行模式的考量，旨在弥合自然语言科学任务与自动化实验室执行之间的鸿沟。

## 1. 引言：连接自然语言与科学实验

### 1.1 科学智能中自动化的必然性

传统的科学发现模式，即“爱迪生式”的试错法，耗时且资本密集，一项基础性发现可能需要长达二十年的时间才能实现。这种缓慢且成本高昂的周转周期，迫切需要颠覆性的策略来加速创新，而“自驱动实验室”正是其中的关键[1]。人工智能（AI）驱动的自动化正在深刻改变实验生物学领域，通过最大限度地减少人工干预，显著提高实验通量、降低成本、减少人为错误，并增强结果的重现性。研究表明，采用AI驱动的自动化可将实验周期缩短60%，试剂消耗减少85%，重现性提高45%，并使新假设的生成增加70%[2]。

科学智能作为一个快速发展的交叉领域，正通过利用机器学习算法和数据驱动方法，加速物理、化学和生物学等学科中自然现象潜在模式的发现。其中，方程式发现是该领域一个特别令人兴奋的前沿，机器学习模型能够直接从数据中推断出潜在的数学关系[3]。这种能力超越了简单的效率提升，它代表着科学方法本身的根本性转变。现代科学问题在规模和复杂性上的指数级增长，例如在药物发现和材料科学中的挑战，已经超出了人类进行手动实验和数据分析的能力。自动化系统的引入，正是为了应对这种复杂性和数据量。一旦自动化，这些系统能够生成海量且一致的数据集。这些数据与AI识别非显而易见模式（如方程式发现、假设生成）的能力相结合，使得过去人类难以实现或不可能实现的突破成为可能。因此，AI并非仅仅加速了旧有流程，它正在催生全新的发现形式。这种范式转变意味着人类科学家的角色将从手动执行转向更高层次的问题定义、AI模型训练以及AI生成洞察的解释，从而从根本上改变了研究范式。

### 1.2 函数调用：使大语言模型与物理实验室交互

函数调用（Function Calling）赋予了大型语言模型（LLM）与外部工具和应用程序接口（API）进行交互的能力，使其超越了简单的聊天机器人功能，能够在现实世界中“执行任务”[4]。通过函数调用，LLM能够根据预定义的函数规范，调用具有详细参数和预期结果的特定函数。这为AI驱动的应用程序注入了急需的结构和可预测性[5]。当用户提示与某个函数的功能相符时，模型能够识别并自动或建议性地发起函数调用，从而检索数据、执行计算或与外部

API交互，而无需持续的人工干预[5]。

LLM从纯文本生成器向能够进行函数调用的“智能体”系统的转变，代表着科学智能领域的一个关键架构转折点。这意味着函数模式（Function Schema）的设计变得与LLM架构本身同等重要，因为它决定了LLM在科学领域内的“感知输入”和“运动输出”能力。如果函数定义不明确或不完整，即使LLM内部的推理能力再强，也可能导致“幻觉”或不正确的函数调用。相反，精心设计的模式能够促成更复杂、多步骤的流程，并优化工作流程[5]。因此，函数模式有效地充当了LLM抽象推理与实验室具体操作之间的接口，使其成为科学智能架构中的核心组成部分。未来，科学智能LLM的成功将高度依赖于领域专家与AI工程师的紧密合作，以精确定义这些函数模式，确保它们准确反映真实实验室设备的能力和限制。

### 1.3 报告目标与结构

本报告旨在为构建面向科学智能的函数调用数据集提供深入的专业知识。报告将围绕以下核心目标展开：

- 盘点当前实验自动化设备及其控制接口：重点关注支持函数调用的仪器。
- 分析工业界与学术界中真实存在的**API**调用系统或案例：揭示实际应用模式。
- 提供可参考的函数调用模式设计建议：助力高质量数据集的构建。
- 汇总现有数据集、开源框架或合作接口信息：为研究者提供资源。
- 总结构建科学智能函数调用数据集的设计要点：为实践提供指导。

报告结构如下：第二部分将探讨学术界的自动化现状，包括常见仪器及其接口、自驱动实验室平台和抽象层。第三部分将分析工业界的自动化实践、商业云实验室及其通信标准。第四部分将深入研究现有科学智能函数调用系统和数据集设计案例。第五部分将展望未来趋势并提出数据集建设的启发。最后，第六部分将总结主要发现并提出建议。

## 2. 学术界：科学实验领域的自动化与可编程性现状

### 2.1 常见的自动化实验仪器及其接口

#### 2.1.1 液体处理系统：能力与程序化控制

液体处理系统是生物、化学实验的核心设备，能够以高精度（F.A.S.T. 系统在0.1 µL时CV小于5%，FLO i8 PD系统在0.5 µL时CV小于5%）吸取和分配各种粘度（0.1 µL至1000 µL）的精确液体体积[6]。它们支持高通量工作流程，广泛应用于ELISA、细胞分析、NGS文库制备和基因分型等领域[6]。

在程序化控制和接口方面，PyLabRobot提供了一个开源、硬件无关的Python接口，用于控制各种液体处理机器人（如Hamilton STAR、Tecan EVO和Opentron OT-2）及其附件[8]。它采用一套通用命令（吸取、分配、吸头拾取、吸头丢弃）和分层资源类（Resource）来表示实验台布局，支持通过REPLs和Jupyter Notebooks进行实时交互控制，并支持并行执行[8]。PyLabRobot还包含一个基于浏览器的模拟器，以降低使用门槛[8]。Formulatrix的F.A.S.T.和

FLO i8 PD液体处理系统均提供了“开放API以实现无缝集成” [6]。尽管具体API类型（如REST、SOAP）在现有资料中未详细说明，但这表明供应商提供了可编程接口，以便集成到更广泛的自动化系统中 [6]。其软件还支持导入.csv文件进行实验设计，暗示了结构化的输入格式 [6]。

Python语言在机器学习和人工智能领域占据主导地位。如果实验室仪器可以通过Python SDK或可封装在Python中的API进行控制，那么LLM生成可执行代码将变得更加容易。PyLabRobot等硬件无关的框架提供了一个通用操作集（如LiquidHandler.aspirate），这意味着LLM无需学习每个Hamilton或Tecan型号的具体细节，从而简化了函数调用模式并降低了LLM任务的复杂性。这种“Python+硬件无关”的趋势将加速LLM在湿实验室中的应用，因为它减少了对深层硬件特定编程知识的需求，从而降低了AI驱动自动化的门槛。PyLabRobot的四个基础命令等通用命令集的开发，对于创建能够跨不同实验室设置进行泛化的鲁棒函数调用数据集至关重要，避免了仅限于特定供应商或仪器的局限性。

### 2.1.2 微流控系统：精密控制与自动化

微流控技术，或称“芯片实验室”技术，能够精确控制和操纵微升至皮升范围的流体，从而在药物活性成分（API）封装和双乳液等应用中实现对颗粒制造过程的增强控制、更高的产率和易于放大 [10]。这些系统能够高效生成微米级颗粒（直径50纳米至50微米），具有极高的单分散性（CV小于1%）和高生成频率（标准系统高达10 kHz，Telos®生产系统可达MHz） [10]。

尽管资料中提到了“精确控制”和“流体操纵” [10]，以及对多功能仪器“灵活接口方法”的需求 [11]，但并未详细说明具体的API类型（如REST、SDK）。然而，微流控的性质（精细控制、复杂序列）强烈暗示程序化控制的必要性。

微流控操作的高精度和复杂序列特性，加上资料中缺乏明确、标准化的API文档，表明当前在为这些系统提供LLM友好的函数模式方面存在空白。这意味着需要更集中地努力来标准化微流控控制接口，或开发更强大的抽象层，以促进LLM的集成。如果接口不标准化或不公开文档，LLM将难以学习如何控制它们。微流控操作的复杂性（例如，微米级的精确流速、温度梯度、混合序列）意味着简单的“吸取/分配”抽象可能不足。每个微流控芯片可能具有独特的控制参数，导致“一台仪器，一个实验”的问题 [11]。这种缺乏标准化、公开文档的API，将提高LLM集成的门槛，从而减缓AI在复杂微尺度实验中的应用。因此，未来在数据集建设方面，应优先为微流控操作开发详细、标准化的函数模式，这可能需要逆向工程现有控制软件或与制造商合作。

### 2.1.3 自动化显微镜：高内涵成像与AI集成

自动化显微镜利用技术和机械来自动执行任务，以提高效率和精度，例如进行多色2D图像采集、识别细胞和检测斑点 [12]。自主显微镜更进一步，通过集成人工智能（AI）和机器学习（ML）进行实时数据分析、决策制定、优化和自适应，从而实现实验的快速迭代并加速科学发现 [12]。

在程序化控制和接口方面，AEcroscoPy是一个Python软件包，它与自主开发的API（PyAE和PyScanner）集成，使研究人员能够将实验计划转化为Python程序，并在显微镜上执行[12]。它提供了高效硬件控制和基于AI的决策制定的基础设施[12]。该框架基于MicroManager和PycroManager的自动化功能构建[13]。

AI/ML直接集成到自动化显微镜工作流程中（如AEcroscoPy）以实现实时决策，突显了对函数调用数据集的迫切需求，这些数据集不仅包含执行命令，还包含反馈回路和条件逻辑。这意味着用于显微镜的LLM需要生成一系列操作，这些操作穿插着数据分析和决策点，从而超越简单的线性协议生成。对“自主实验”和“实时数据分析和决策制定”的强调[12]表明显微镜不仅仅是遵循脚本，它还在进行自适应。这需要函数能够：a）获取数据，b）处理数据（例如，识别细胞/斑点），以及c）决定下一次获取（“获取-处理-决策”机制[13]）。这是一种比典型液体处理协议更复杂的控制流。对显微镜中实时自适应实验的需求推动了支持动态控制和反馈的API的开发。这反过来要求函数调用模式能够表示条件分支、迭代循环和基于实时数据的动态参数调整，这比简单的顺序命令执行更为复杂。因此，用于显微镜LLM的数据集应包含自适应协议的示例，其中LLM生成的不仅仅是静态的步骤序列，而是决策树或包含条件语句和循环的程序，以反映“获取-处理-决策”的范式。

#### 2.1.4 高通量筛选（HTS）系统：规模与数据生成

高通量筛选（HTS）在药物发现中采用，通过自动化、机器人技术、高度微型化的检测形式（从96孔到1536孔微孔板）以及先进的检测技术（如荧光、发光和质谱）快速测试大量的化合物、生物分子或基因敲除库[14]。其目标是识别“命中”化合物，并生成可进行统计分析的定量读数，从而深入了解作用机制、潜在脱靶效应、毒性和药效学特性[14]。例如，Sartorius iQue® 3 HTS细胞仪平台能够在5分钟内筛选一个96孔板，并支持多种细胞参数的多重检测[15]。

HTS本质上依赖于自动化和机器人技术。AI算法被整体集成到多个阶段，从检测优化到实时监测和质量控制[14]。Sartorius iQue® 3平台具有“机器人集成能力”，适用于高通量应用[15]。尽管资料中未详细说明具体的API类型，但对AI系统实时监测性能、标记异常并实现动态调整的描述，暗示了程序化访问和控制的存在[14]。

HTS系统庞大的规模和对实时数据分析的需求表明，用于HTS的LLM函数调用不仅涉及协调仪器操作，还涉及与复杂的数据管理和分析管道集成。这意味着HTS的函数模式必须包含数据采集、实时分析触发器，以及潜在的、用于自适应筛选的反馈机制参数。对“实时分析”和“动态调整”的强调[14]表明系统并非简单地执行预编程序列，而是持续监测数据并做出决策。这需要函数不仅能够命令仪器（例如，run_plate_screen(plate_id, assay_type)），还能够与数据处理单元交互（例如，analyze_hts_data(run_id), flag_anomalies(data_stream)），并可能根据分析结果触发后续操作（例如，retest_wells(well_list)）。对高通量和快速识别有前景候选化合物（效率）的需求，推动了对实时、AI驱动的自适应控制的需求。这反过来要求函数调用接口能够处理复杂的数据流、与分析工具集成，并支持实验循环中的动态决策。因此，用于HTS的函数调用数据集应包含

仪器控制命令以及数据处理和决策制定函数，以反映现代HTS工作流程的完整闭环性质。

表1：学术界自动化仪器：可编程性与接口类型

| 仪器类型 | 示例仪器/平台 | 主要功能 | 程序化控制支持 | 接口类型（示例） | 备注/特点 |
|---|---|---|---|---|---|
| 液体处理系统 | PyLabRobot | 精确吸取和分配液体，高通量移液 | 是 | Python SDK (开源) | 硬件无关，通用命令集，支持交互式控制和模拟器 |
| 液体处理系统 | Formulatrix F.A.S.T. / FLO i8 PD | 低容量/任意粘度液体处理，高精度，高通量 | 是 | 开放API (类型未详述) | 强调“无编程”和自动生成指令，支持导入.csv文件 |
| 微流控系统 | Dolomite | 精确流体控制，API封装，颗粒生成 | 是（暗示） | 未明确API类型 | 高精度，复杂序列操作，可能需要更明确的抽象层 |
| 自动化显微镜 | AEcroscoPy | 自动化图像采集，AI驱动实时决策 | 是 | Python SDK (PyAE, PyScanner) | 基于Python，支持AI/ML集成，实现“获取-处理-决策”循环 |
| 高通量筛选系统 | Sartorius iQue® 3 | 快速筛选化合物库，多参数细胞分析 | 是（暗示） | 机器人集成能力 | 强调实时分析和动态调整，需要与数据管理集成 |

此表旨在提供对常见学术仪器、其主要功能、已报告的程序化控制方法（API/SDK）以及典型接口类型的结构化概览。此表可作为研究人员快速了解仪器可访问性，以进行LLM集成的参考。通过此表，用户可以迅速识别具有高程序化可访问性（例如，Python SDK）的仪器，这些仪器是初期函数调用数据集开发的理想候选者，同时也指出了需要更多接口标准化或抽象的领域。

## 2.2 “自驱动实验室”与自主实验平台

### 2.2.1 高校主导的倡议

多伦多大学的机器人化学家/加速联盟：旨在将功能材料的发现和优化成本与时间降低十

倍。该项目将人工智能与机器人平台相结合，实现材料的自主发现和优化，涵盖机器学习、材料建模、机器人技术、自动化和计算机视觉等多个学科[16]。例如，“溅射器”（Sputtertron）是一种自驱动实验室设备，能够自主表征和分析新合金的电子特性[17]。这项倡议获得了2亿美元的联邦拨款，凸显了其在研究领域的重要投资[17]。

卡内基梅隆大学的**Coscientist**（材料集成设计实验室）：由Gabe Gomes团队设计，Coscientist利用LLM（如GPT-4、Claude）从自然语言提示开始，设计、规划并执行化学实验。它能够搜索互联网、文档数据和其他可用资源，综合信息，选择实验方案，并通过机器人API将其发送给自动化仪器执行[18]。该系统已成功展示了规划已知化合物的化学合成、导航硬件文档以及在云实验室中控制液体处理仪器的能力[18]。卡内基梅隆大学将于2024年初与Emerald Cloud Lab合作，开设首个大学云实验室，提供200多台设备的访问权限[18]。

伯克利大学的自动化学实验室：尽管在现有资料中未详细说明，但可推断其为类似的自驱动实验室倡议。

这些自驱动实验室倡议在投资规模和多学科性质（结合机器学习、机器人技术、材料科学、计算机视觉）上都非常显著，突显了真正自主科学发现的复杂性和高价值。对于函数调用而言，这意味着LLM需要在更广泛的“科学智能体”框架内运行，不仅要集成仪器控制，还要整合知识检索、实验设计和实时数据解释。自驱动实验室的目标超越了简单的自动化，它追求的是“自主发现和优化”[16]。Coscientist能够“搜索互联网、文档数据……综合信息并选择实验方案”[18]，这表明LLM正在充当一个复杂的智能体。它不仅仅是接收函数调用，而是根据高层次目标和动态信息来生成一系列调用。对加速科学发现的追求，推动了LLM需要具备复杂推理、知识整合和自适应规划的能力。这表明，用于此类系统的函数调用数据集必须超越简单的命令-参数对，包含多步骤推理、工具选择和基于科学目标的动态协议生成等示例。因此，“AI科学家”的概念[19]是最终目标，即LLM能够提出假设、将其转化为协议、执行并循环迭代地优化模型。这要求函数调用嵌入到更大的智能体架构中。

### 2.2.2 自主科学发现的架构范式

自驱动实验室将AI与机器人平台相结合，以实现自主发现和优化[16]。它们的核心在于实现计算建模与实验验证之间的闭环[16]。像Coscientist这样的系统，利用LLM从简单的自然语言提示开始，通过机器人API控制仪器，执行整个实验过程，包括搜索和导航硬件文档[18]。此外，一个自动化协议翻译的框架，通过三阶段工作流（语法、语义、执行）逐步构建协议依赖图（PDG）[20]。这旨在弥合人类可理解的自然语言（NL）协议与机器可执行步骤之间的鸿沟[20]。

自驱动实验室的核心架构挑战在于“自然语言到执行”的鸿沟，这需要复杂的中间表示和多阶段翻译过程。这意味着在这种背景下的LLM函数调用并非一步到位，而是一个多智能体系统，其中LLM可能生成高层次计划，然后由其他模块或专门的LLM智能体将其分解并翻译成低层次的仪器命令。对“闭环”的强调[16]以及Coscientist从“自然语言提示”到“机器人API”的转换[18]，再到协议翻译框架[20]，都表明直接的自然语言到API调用通常不足以完成任

务。在这些流程中存在一个结构化的中间层。LLM可能生成一个高层次的“语义协议”，然后需要对其进行解析、验证，并翻译成特定的仪器命令。这指向了任务的层次化分解。科学实验固有的复杂性和多模态特性，决定了LLM集成需要采用分层架构方法。这意味着LLM不太可能直接发出低层次命令，而是与一个抽象层交互，该抽象层负责翻译和编排，可能通过涉及不同“工具智能体”的函数调用链来实现。因此，数据集设计应考虑这些层次结构：某些函数调用可能是高层次的（例如，run_synthesis_protocol），而另一些则是低层次的（例如，aspirate_volume）。训练LLM理解并生成这些嵌套或链式调用，可能包含中间推理步骤，将至关重要。

### 2.3 实验设备抽象与控制框架（学术界）

#### 2.3.1 设备抽象层的概念

OSGi设备抽象层（DAL）规范为应用程序开发人员提供了一个统一的接口，用于与连接到网关的传感器和设备进行交互，从而抽象出协议特定的细节 [21]。它将物理设备映射到Device服务，这些服务也可以表示纯软件单元以进行模拟 [21]。设备通过functions暴露其操作和属性 [21]。

设备抽象层（如OSGi DAL）的规范化对于LLM集成超越单个仪器至关重要。这意味着函数调用模式理想情况下应映射到这些抽象的、统一的接口，而非原始的、特定于供应商的命令，从而提高互操作性并减轻学习多样化低层次API的负担。LLM受益于一致、结构化的输入。如果每个“液体处理系统”或“显微镜”都通过标准化的Device服务和functions（如OSGi DAL所定义）来呈现其能力，那么LLM生成正确函数调用的任务将大大简化。它无需知道是Hamilton还是Tecan的设备，只需要知道抽象的LiquidHandler.aspirate函数。这减少了LLM需要学习的设备特定知识的“长尾效应”。有效的设备抽象能够带来标准化的函数模式，进而改善LLM的泛化能力并减少新设备所需的训练数据。因此，在设计函数调用数据集时，优先考虑映射到（或可以映射到）通用抽象层的函数，将产生更具泛化性和可重用性的LLM智能体。这也暗示了LLM生成新的抽象层或在现有抽象层之间进行映射的潜力。

#### 2.3.2 开源自动化框架

**PyLabRobot**：一个开源、跨平台的Python接口，用于液体处理机器人及其附件。它提供了一套通用命令（吸取、分配、吸头拾取、吸头丢弃）和实验台布局表示，能够控制各种附件设备 [8]。它采用分层架构，包括Interface Layer（例如，LiquidHandler）和Execution Layer（针对特定硬件的后端），最大限度地减少了后端职责，便于集成新仪器 [8]。它支持交互式控制，并包含一个模拟器 [8]。

**ChemOS**：一个可移植、模块化、多功能的开源Python软件包，用于操作自主实验室 [1]。它为自驱动实验室提供了结构化层，促进了自动化设备的集成，实现了远程控制，并包含了最先进的机器学习方法 [1]。它可以在不同程度的自主性下运行，从无人监督的实验到人机协作 [1]。其GitHub仓库是公开的 [1]。

**ESCALATE**（实验规范、捕获和实验室自动化技术）：一个本体论框架和开源软件包，为人类和机器可读的实验规范以及全面的（元）数据捕获提供了抽象层 [23]。它简化了数据收集，并与机器学习集成 [23]。它使用“模板”来定义实验模式，并生成用于预期实验室操作的“模型”。程序化控制通过基于网络的API和命令行程序实现，具有灵活的输出格式（例如，用于Hamilton Microlab NIMBUS4机器人控制文件的XLS） [23]。一个实验提交和验证API正在开发中 [23]。

开源、基于Python的自动化框架（PyLabRobot、ChemOS、ESCALATE）的存在和积极发展，是科学智能函数调用的关键推动因素。这些框架提供了现成的、结构化的接口，LLM可以将其作为目标，从而减少了从零开始构建仪器驱动程序的需要。它们的开源性质也促进了函数模式和数据集的社区驱动开发。Python在AI领域广泛应用，使得这些框架可以直接被LLM生成的代码使用。它们在一定程度上实现了标准化（例如，PyLabRobot的通用命令），从而简化了LLM的函数模式设计。ESCALATE的本体论框架和实验规范“模板” [23] 直接指导了协议结构化数据的表示方式，这对于LLM训练非常理想。开源特性促进了社区贡献，可能带来更丰富的函数定义和协议示例生态系统，可用于数据集生成。PyLabRobot的模拟器 [8] 对于在没有物理硬件的情况下测试LLM生成的协议具有不可估量的价值，加速了函数调用智能体的开发和调试。开源框架、Python语言和结构化接口的结合，降低了LLM集成的门槛和数据集创建的复杂性，从而加速了AI驱动实验室自动化的发展。因此，数据集开发者应将这些框架作为基础层，提取其内部函数定义和协议结构，以创建高质量的训练数据。向这些项目贡献LLM生成的协议或函数模式，可以进一步加速该领域的发展。

## 3. 工业界：自动化现状与设备集成模式

### 3.1 主流生物医药/合成化学公司及其自动化生态系统

#### 3.1.1 Insitro：机器学习驱动的药物发现自动化

Insitro致力于创建由机器学习驱动的闭环药物发现系统 [19]。该公司利用AI驱动的实验室自动化来设计、执行和分析实验，最大限度地减少人工干预，从而显著提高通量 [2]。其方法结合了AI对组学数据进行目标识别、生成化学模型用于新分子生成、预测药物特性和潜在毒性，以及优化先导化合物 [2]。AI系统能够自主探索化学空间并合成候选药物 [2]。

尽管资料中未提供具体的API细节，但对“机器学习驱动的闭环系统”和AI系统自主探索化学空间的描述，暗示了深度的程序化控制和用于编排复杂工作流程的内部API。资料也指出“碎片化的工具链和API”是行业普遍存在的问题 [19]。

Insitro对“闭环药物发现”的强调表明，工业界的科学智能应用需要计算设计、自动化执行和实时数据反馈的无缝集成。这意味着工业环境中的函数调用不仅必须支持单个仪器操作，还必须支持整个迭代的“设计-构建-测试-学习”（DBTL）循环，这通常涉及专有的内部API。对“闭环”的强调意味着一个阶段的输出（例如，实验数据）直接作为下一阶段的输入（例如，AI模型优化、新实验设计）。这不仅仅是控制一台仪器，而是控制一个涉及多个步骤、计算模型和迭代优化的过程。资料中提及的“碎片化的工具链和API” [19] 表明，集成这些闭环系

统是一个重大的工程挑战，通常需要定制API开发。对加速药物发现的需求（业务目标）推动了高度集成、闭环自动化的发展。这促使公司开发复杂的内部API和编排层，这些是LLM函数调用的主要目标，但也因其专有性而带来挑战。因此，用于工业应用的函数调用数据集应反映这些复杂的多阶段工作流程，包括数据摄取、模型推理和自适应实验设计等功能，而不仅仅是基本的仪器控制。数据集创建者面临的挑战是访问和标准化这些通常是专有的工业API。

### 3.1.2 XtalPi：AI赋能固态化学与实验增强

XtalPi将计算化学和人工智能相结合，以先进的资源和专业支持，赋能药物固态研究，包括晶体结构预测（CSP）、虚拟筛选和形态预测 [24]。其XtalGazer™平台将计算方法与实验技术相结合，进行固态分析，利用AI驱动的精度预测稳定晶型，并实现24/7自动化多晶型筛选 [25]。他们声称，“AI增强方法”与传统方法相比，能带来更高的命中率和更少的实验次数 [25]。

CSP服务利用“灵活的API”来扩大研究空间 [24]。他们的虚拟筛选平台通过计算目标API与各种溶剂之间的结合倾向来简化最佳共晶形成剂的选择，从而减少实验验证周期 [24]。虽然CSP本身“不需要API或实验设施” [24]，但整个XtalGazer™平台结合了“AI的理性设计与自动化的快速执行” [25]，暗示了内部自动化API的存在。

XtalPi的模型突出了通过AI和自动化将计算模拟与物理实验相结合的集成方式。这意味着该领域的函数调用将扩展到直接的仪器控制之外，包括用于计算建模、数据模拟和优化算法的功能，这些功能随后为物理实验提供信息。对“计算化学与AI”结合“实验技术”的观察 [24]，以及CSP不需要物理API但整个平台使用“灵活API” [24] 的事实，表明LLM可能在生成物理实验室设备命令之前，与运行模拟、访问化学性质数据库或执行优化算法的函数进行交互。物理实验室随后用于“快速执行”和“自动化多晶型筛选” [25]。将体外和体内过程（计算预测指导物理实验）相结合的需求，要求函数调用涵盖计算工具（例如，run_csp_simulation、predict_morphology）和物理实验室操作（例如，perform_crystallization、analyze_powder_sample）。因此，用于此类集成工作流程的函数调用数据集应包含计算和物理操作的函数，以及LLM如何将这些调用链接起来以实现科学目标的示例。

### 3.1.3 Benchling：云端研发平台与LIMS及仪器集成

Benchling是一个面向生命科学研发的云端平台，提供电子实验记录本（ELN）、实验室信息管理系统（LIMS）、分子生物学工具、实验室自动化、数据分析和机器学习功能 [26]。Benchling提供了LIMS自动化和仪器集成功能，旨在通过自动同步仪器运行数据来简化工作流程、减少手动数据输入并提高数据完整性 [27]。它允许用户通过基于数字样本创建机器人编程指令来“编排机器人” [27]。

Benchling的开发者平台具有**REST API**、事件触发集成和内置数据仓库，提供了一个用于无缝集成的集中式平台 [27]。TeselaGen通过Benchling API导入/导出序列、自动化注释/分析、增强DNA组装工作流程并同步实验数据/元数据 [26]。

Benchling全面的云端平台，凭借其强大的REST API和事件触发集成，展示了将实验室自动化与数据管理相结合的成熟模式。这意味着此类平台的函数调用将不仅涉及仪器控制，还涉及广泛的数据管理（例如，样品追踪、元数据记录）和工作流程编排，充分利用平台的API生态系统。LIMS/ELN不仅是控制仪器，更是管理实验的整个生命周期，从样品注册到数据分析和报告。REST API和事件触发集成[27]意味着LLM不仅可以向仪器发送命令，还可以从仪器接收反馈（通过事件）、更新样品库存、记录结果并触发下游分析。这是一种比简单命令执行更丰富的交互模型。生命科学研发对端到端数据可追溯性和合规性的需求推动了Benchling等集成平台的发展。这反过来又产生了对函数调用能力的需求，这些能力能够与研究工作流程的所有方面进行交互，包括数据管理和元数据捕获。因此，函数调用数据集应包含与数据管理相关的函数（例如，register_sample、update_inventory、log_experiment_data），而不仅仅是直接的仪器控制。这反映了受监管和复杂科学研究的现实。

### 3.1.4 Recursion：高通量生物学与AI驱动的药物发现

Recursion专注于大规模的AI驱动药物发现和开发，其核心是全球最大的专有生物和化学数据集之一（约60 PB）[28]。其“Recursion操作系统（OS）”利用自动化湿实验室、机器人技术和计算机视觉，每周捕获数百万个细胞实验（每周高达220万个样本）[28]。所有结果都会反馈到平台中，形成一个持续改进的反馈循环[29]。他们利用CRISPR进行大规模细胞扰动，并开发了创新的细胞培养方法[28]。他们还拥有一个名为LOWE的LLM药物发现智能体[29]。

该系统被描述为“持续改进的反馈循环”，并使用“机器人技术和计算机视觉”[29]。尽管资料中未提供明确的API细节，但其规模和闭环性质强烈暗示存在高度复杂的内部API和编排层。LOWE（一个LLM智能体）的提及表明LLM可以直接与该系统交互[29]。

Recursion的“持续改进反馈循环”和大规模数据生成代表了AI4Science闭环发现的典范。这意味着此类系统的函数调用必须深入集成到整个研究生命周期中，使LLM不仅能够执行任务，还能从结果中学习，并自主提出新实验以完善生物和化学“图谱”。目标是“构建生物学和化学图谱”，并“快速识别新靶点和设计高度优化的分子”[29]。反馈循环意味着LLM不仅仅是执行，它还是科学发现引擎的一部分。这需要函数能够：a）触发大规模并行实验，b）摄取和处理PB级数据，c）更新内部知识图谱/模型，以及d）根据模型洞察生成新假设或实验设计。LOWE充当了LLM与这个复杂系统之间的接口。大规模绘制生物和化学图谱的雄心，推动了对全自动化、AI驱动实验平台的需求。这要求函数调用能力能够处理极端的并行性、复杂的数据管道以及在持续发现周期内的智能、自主决策。因此，用于此类高级LLM智能体的数据集不仅需要包含显式函数调用，还需要包含科学问题的背景、实验目标以及驱动迭代学习过程的预期结果。这使得LLM能够生成完整的科学工作流程，而不仅仅是单个步骤。

### 3.1.5 Synthace：实验数字化平台

Synthace是一个面向生命科学研发的数字化实验平台，使科学家能够通过易于访问的界面

设计、执行和分析实验[30]。Synthace提供了“无代码实验设计”，通过拖放界面自动生成各种仪器的设备指令[30]。它与多种实验室设备集成，允许在不同机器上运行相同的检测[30]。当设计发生变化时，它会动态更新自动化指令，并在云端生成、聚合和结构化所有数据和元数据，将其与原始设计关联起来[30]。

该平台强调“无代码”设计和指令的自动生成，这表明存在一个高层次的抽象层[30]。尽管资料中未明确提及用于直接程序化控制的公共API，但该平台“自动将协议转换为自动化指令”的能力暗示了一个内部的、结构化的表示，可以暴露或模拟以进行函数调用[30]。

Synthace的“无代码”实验室自动化方法，能够从高层次设计动态生成仪器指令，这表明其内部抽象层功能强大。这意味着LLM的函数调用可以针对这种高层次的设计语言或生成的中间指令，为LLM提供一个比低层次设备命令更简单的目标。如果Synthace能够将拖放设计转换为可执行指令，则意味着存在一个明确定义的实验协议内部表示。LLM可以潜在地生成这种高层次的“无代码”设计（例如，拖放工作流程的结构化JSON表示），然后由Synthace将其编译为仪器命令。这充当了更高层次的函数调用，抽象了低层次的仪器细节。对用户友好、灵活的自动化工具的需求推动了鲁棒内部抽象层的创建。这些抽象层，无论是作为API暴露还是从其“无代码”输出中推断，都成为LLM函数调用的理想目标，因为它们通过在更高的语义级别操作来简化LLM的任务。因此，数据集设计可以侧重于生成这些高层次的“协议设计”或“工作流程定义”，然后可以由现有自动化平台进行翻译，而不是直接生成低层次的仪器命令。这利用了现有基础设施，并简化了LLM的角色。

### 3.2 商业云实验室平台与远程函数调用

#### 3.2.1 Strateos：自动化即服务与协议定义

Strateos通过远程访问智能实验室和私有实验室控制软件提供“自动化即服务”[32]。科学家可以通过网络浏览器远程设计、运行和分析实验[32]。Strateos提供“安全API”和“文档”，用于工作流程设计、编排、集成运行/样品/试剂管理和数据分析[33]。用户可以定义协议设计参数并提交运行请求[33]。执行实验的数据会近乎实时地摄取[33]。Strateos最初是Transcriptic，后者开发了Autoprotocol[34]。

资料中未明确其API的数据格式，但鉴于其源自Transcriptic，Autoprotocol（JSON序列化[35]）很可能是其协议定义和数据交换的核心部分[33]。

Strateos从Autoprotocol的开发者发展成为一个提供“自动化即服务”的完整平台，并提供安全API，这证明了结构化协议语言在远程实验室控制中的商业价值。这意味着Autoprotocol的JSON模式[35]是科学智能函数调用数据集的有力候选者，因为它已为机器可读和执行而设计。Strateos提供“自动化即服务”和“安全API”[32]，并且源自开发Autoprotocol的Transcriptic[34]。Autoprotocol被明确设计为一种“用于指定实验协议的语言……人类和计算机都能理解”，并以JSON格式序列化[34]。这与用户构建函数调用数据集以微调LLM以“生成实验协议”的目标直接吻合。如果Strateos（一家领先的云实验室）使用或

基于Autoprotocol，那么其结构和命令是实验室自动化函数模式的经过验证的真实世界示例。云实验室（如Strateos）的商业成功验证了机器可读协议语言的实用性。这为设计LLM函数调用模式提供了明确的先例和丰富的灵感来源，这些模式能够将自然语言转换为可执行的实验室协议。因此，Autoprotocol应作为用户数据集中“函数模式”设计的主要参考，尤其是在协议生成任务方面。

### 3.2.2 Emerald Cloud Lab (ECL)：综合性远程实验室控制

Emerald Cloud Lab（ECL）被誉为“世界上第一个自动化、集成、综合性、远程控制、软件可编程的临床前生物制药研发实验室”[36]。它通过统一的软件界面ECL Command Center®，提供对湿实验室操作的全面控制，从样品制备到实验执行和数据分析，均可远程访问[37]。该设施24/7运行，拥有200多种仪器型号[37]。

用户通过专门的网页应用程序或API提交协议[38]。实验执行后，数据通过API自动上传到云实验室[38]。所有技术都“可脚本化并可一键重现”[37]。Command Center提供了4500多个数据分析和可视化功能[37]。每个参数都通过命令精确指定，其输出被编译成“机器可读的符号实验室语言（SLL）软件程序”[36]。SLL最大限度地减少了人为错误，并支持协议调试[36]。

ECL专有的符号实验室语言（SLL）及其庞大的4500多个功能库，代表了一个高度复杂但封闭的函数调用生态系统。SLL是机器可读的，并精确指定每个参数[36]，这表明其功能分解和参数化达到了理想的深度，非常适合LLM训练，即使直接访问SLL受限。这暗示了需要对其他实验室系统进行逆向工程或推断类似的细粒度函数定义。ECL是一个综合性云实验室，拥有“统一的软件界面”和“超过4500个功能”[37]。它使用“机器可读的符号实验室语言（SLL）”[36]。SLL作为实验室自动化的内部领域特定语言（DSL），“机器可读”且“精确指定每个参数”[36]，这与用户创建函数调用数据集的目标直接平行。4500多个功能[37]表明了其功能集的细粒度和全面性。虽然SLL是专有的，但这展示了一个完整的实验室自动化函数集及其参数应达到的细节和广度。对“民主化昂贵实验室设备访问”和“标准化实验执行”的需求[38]推动了这种全面、集中控制语言的开发。这为LLM集成提供了丰富但专有的蓝图，说明了完整的实验室自动化函数集及其参数应该是什么样子。因此，尽管SLL本身可能无法直接使用，但其结构和粒度应启发数据集中的函数模式设计。用户应力求以相似的精度和细节定义函数，涵盖广泛的操作，并可能从ECL公开描述的功能中推断出通用模式。

### 3.2.3 Arctoris：AI驱动的药物发现自动化

Arctoris运营着一个专注于药物发现的全自动化实验室，其中AI系统协助设计实验并生成协议[19]。他们最近收购了Eli Lilly的Science Studio生物学平台，使其实验室规模翻倍，成为全球最大的机器人药物发现平台之一[39]。

AI系统协助设计实验并生成可执行的协议[19]。收购Eli Lilly的“AI控制机器人，研究人员可通过云端访问的闭环系统”[39]进一步巩固了其AI驱动、云端可访问的自动化模式。尽管资料中未提供明确的API细节，但描述暗示了用于AI交互和协议生成的强大程序化接口。

Arctoris的快速扩张以及对云实验室环境中AI驱动“协议生成”的关注，突显了LLM将科学意图转化为可执行实验室指令的商业可行性和需求。这强化了用户项目的核心前提，并表明成功的函数调用数据集将使LLM能够生成完整、可执行的协议，而不仅仅是单个命令。Arctoris使用AI“设计实验并生成协议”[19]，并扩展了其自动化实验室[39]。对“AI系统协助设计实验并生成协议”[19]的关注，直接符合用户微调LLM以“自动调用实验器材、控制实验流程或生成实验协议”的目标。这表明LLM的输出应该是一个协议（一系列函数调用），而不仅仅是单个函数调用。对加速药物发现的商业需求，推动了AI能够自主生成和执行实验协议的平台的发展。这要求函数调用能力能够表示和编排整个实验工作流程。因此，函数调用数据集应优先包含多步骤协议的示例，包括条件逻辑和错误处理，而不仅仅是孤立的函数调用。

表**2**：商业云实验室平台：**API**访问与数据交换

| 平台 | 核心服务 | **API/**程序化访问 | 协议定义语言**/**格式 | 数据交换格式 | 自动化关键功能 |
|---|---|---|---|---|---|
| Strateos | 自动化即服务，远程智能实验室 | 是 (安全API) | Autoprotocol (JSON序列化) | 近实时数据摄取 | 远程控制，工作流程编排，样品/试剂管理 |
| Emerald Cloud Lab (ECL) | 综合性远程实验室控制 | 是 (API, Web应用) | 符号实验室语言 (SLL) | 自动上传，ALCOA+标准，4500+功能 | 24/7运行，200+仪器，所有技术可脚本化 |
| Arctoris | AI驱动药物发现自动化 | 是（暗示） | AI生成协议 | 未明确 | AI辅助实验设计，协议生成，机器人控制 |

此表旨在总结商业云实验室平台的主要特点，重点关注它们如何以程序化方式暴露自动化能力以及如何处理数据。此表将突出其API策略和数据格式的共性和差异，为设计可泛化的函数调用数据集提供信息。通过此表，可以发现，尽管具体实现方式各不相同，但对结构化、机器可读协议和全面数据捕获的潜在需求是普遍存在的。这强化了函数调用数据集模式设计的重要性。

### 3.3 自动化实验设备通信的工业标准

#### 3.3.1 OPC UA（开放平台通信统一架构）

OPC UA提供了平台无关、安全的数据交换和可伸缩性，适用于工业环境[40]。它提供标准化的通信协议和可互操作的数据模型，从而简化系统集成[40]。OPC UA通过实现信息交换和互

操作性，正在工业自动化领域产生影响[40]。它通过提供集中式、语义化的数据存储，并在云端保留信息模型的上下文，从而促进了数字孪生、AI和元宇宙等新应用的发展[40]。它允许直接的控制器到控制器（C2C）通信，并将服务器直接嵌入到边缘设备中[40]。它还内置了安全机制[40]。

OPC UA的标准化互操作性可以加速使用AI的IT和云应用程序[40]。其数据建模规则可以创建复杂的信息模型[40]。

OPC UA对语义数据模型和平台无关、安全通信的关注，与科学智能高度相关，特别是对于将多样化的实验室仪器集成到统一的数据和控制平面中。这意味着支持OPC UA的仪器的函数调用可以利用这些语义模型来生成更具上下文感知性和鲁棒性的命令，超越简单的参数传递。OPC UA提供了“平台无关”、“安全数据交换”、“可互操作数据模型”和“语义数据存储”[40]，并且明确提及用于“AI”应用[40]。LLM受益于结构化、语义丰富的信息。如果仪器通过OPC UA信息模型暴露其能力和状态，LLM可以比通过扁平API模式更深入地“理解”仪器的上下文。例如，LLM可以推断出start_pump函数需要flow_rate参数，并且flow_rate在OPC UA模型中定义了特定的单位和范围。这使得参数生成和验证更加智能化。语义互操作性标准（如OPC UA）的采用，将使LLM能够以更高层次的理解与实验室设备交互，从而产生更可靠、更符合上下文的函数调用。因此，函数调用数据集设计应考虑如何整合仪器能力和数据类型的语义信息，例如通过将其映射到标准化本体论，或通过使用类似OPC UA的元数据来丰富函数描述。

### 3.3.2 SiLA2（实验室自动化标准）

SiLA2是一种专为实验室自动化设计的开源通信协议[41]。它基于HTTP/2，采用类似REST的范式，并使用ProtocolBuffer数据结构[41]。它侧重于功能而非设备类型，表示设备行为而非底层状态[41]。SiLA2简化了来自不同制造商的实验室仪器的集成，消除了专有系统带来的挑战，并促进了适应性和可伸缩性[42]。它能够将实验室仪器连接到云端，并简化安全证书管理[41]。

SiLA2将“功能”（Features）定义为SiLA服务器（仪器）的能力，这些功能暴露“命令”（Commands，带有参数的操作）和“属性”（Properties，可读或可注册的数据点）[41]。当命令执行后，它会返回完整且可追溯的数据[41]。SiLA Discovery机制允许客户端自动连接并发现SiLA服务器的功能[41]。

SiLA2的开放、标准化和消息驱动的方法使其非常适合LLM函数调用。其对“功能”、“命令”和“参数”的清晰定义直接转化为一个鲁棒的函数模式[41]。该标准强调在命令执行后返回“完整且可追溯的数据”，这对于闭环系统中的LLM至关重要[41]。

SiLA2是科学智能函数调用最直接、最有前景的标准，因为它专为实验室自动化而设计，并提供了一种清晰、结构化的方式来将仪器功能暴露为可调用函数。其开放性以及对“功能”和“命令”的关注，为设计全面且可互操作的函数调用数据集提供了现成的蓝图。SiLA2是

“实验室自动化的开源通信协议” [42]。它定义了“功能”、“命令”和“属性” [41]。SiLA2的“功能”（能力组）和“命令”（带参数的可调用动作）概念模型直接映射到函数调用范式。该标准对“标准化状态”、“互操作性”和“消息驱动”通信的强调 [41] 意味着在SiLA2下定义的函数本质上是结构良好的，适合LLM使用。命令执行后返回“完整且可追溯的数据” [41] 意味着LLM可以接收结构化反馈，这对于多轮交互和错误处理至关重要。SiLA2专为实验室自动化而设计，直接降低了为LLM定义函数模式的复杂性。通过与SiLA2对齐，函数调用数据集可以实现更高的互操作性，并提高其在不同实验室仪器和平台之间的可重用性。因此，在设计函数模式时，应大量参考SiLA2规范（规范A部分、B部分 [41]），如果可行，甚至可以采用SiLA2的ProtocolBuffer数据结构用于参数和返回值。这将使数据集与新兴的行业标准高度对齐。

### 3.3.3 ISA-88（批次控制）与MODBUS

**ISA-88**：一系列批次过程控制标准，为制药和化工等行业的批次处理带来了清晰性、结构性和一致性 [44]。它标准化了术语，实现了模块化设计，并高效管理配方 [44]。它定义了用于组织和交换批次过程信息的数据结构（ISA-88.00.02-2001），以及通用和站点级配方模型（ISA-88.00.03-2003） [44]。它还提供了用于构建批次数据以实现可追溯性和完整性的框架（ISA-88.00.04-2006） [44]。

**MODBUS**：工业自动化系统中广泛使用的通信协议，用于可编程逻辑控制器（PLC）和传感器等设备之间的通信 [45]。它作为主从协议运行，定义了用于对寄存器或数据点进行特定操作的功能码（例如，读取保持寄存器、写入单个寄存器） [45]。

在实验室自动化/AI中的作用和潜力：

- **ISA-88**：其对配方管理和批次过程控制的结构化方法 [44] 为LLM生成复杂、多步骤的实验协议提供了强大的概念框架。ISA-88中的“配方”概念类似于LLM可能生成的高层次实验协议。
- **MODBUS**：尽管比OPC UA或SiLA2更低级和简单，但MODBUS定义的“功能码” [45] 是原始函数调用的直接示例。它代表了LLM可能需要理解的非常细粒度的设备交互基础层，或者更可能通过在其之上构建的抽象层进行交互。

尽管OPC UA和SiLA2更直接适用于现代高层次实验室自动化和LLM集成，但ISA-88和MODBUS突显了理解工业控制中不同抽象层的重要性。ISA-88为协议结构（配方）提供了有价值的模型，而MODBUS则体现了低层次设备命令。这意味着一个全面的函数调用数据集可能需要包含不同粒度的函数，从高层次的配方步骤到低层次的寄存器操作，或者专注于抽象这些低层次的接口。ISA-88侧重于“批次过程控制”和“配方” [44]，而MODBUS定义了PLC和传感器的“功能码” [45]。ISA-88的“配方”概念与实验协议直接平行。LLM可以被训练生成符合ISA-88标准的配方，然后由批次控制系统执行。MODBUS虽然非常低级，但展示了最基本的函数调用形式：用于对寄存器进行特定操作的代码。这显示了程序化控制的基础元素。工业自动化标准的历史发展提供了不同抽象层次的谱系。对于LLM函数调用而言，这意味着虽然SiLA2等高层次标准简化了任务，但理解其底层原理（如ISA-88的批次控制逻辑

或MODBUS的基本寄存器访问）可以为设计更鲁棒、更全面的函数模式提供信息。因此，数据集可以包含“配方”（高层次函数）的示例，这些配方在内部分解为低层次操作序列，从而展示层次化函数调用。这也指出了LLM在不同抽象层之间进行映射的挑战。

## 4. 科学智能函数调用系统与数据集设计案例

### 4.1 现有科学领域函数调用数据集与API模式

#### 4.1.1 Autoprotocol：实验协议规范语言

概述：Autoprotocol是一种开放标准语言，用于以精确、明确且人类和计算机都可理解的方式指定实验协议 [34]。它最初由Transcriptic（现为Strateos）开发，旨在定义用于远程机器人自动化（“云研究”）的实验 [34]。

结构与模式：协议使用**JSON**进行序列化 [34]。它们是线性的指令序列，不包含运行时可评估的分支逻辑或循环结构 [35]。关键约定包括“维度值”（如“50:second”、“25:microliter”等数量）、用于容器的“refs”以及用于生成数据的“datarefs” [35]。容器和孔位使用斜杠语法引用（例如，my_plate/A1） [35]。

命令/操作：示例包括Distribute（体积、来源、目的地）、Centrifuge（板、持续时间、力）、Absorbance读数（孔位、波长） [35]。autoprotocol-python库是一个用于生成Autoprotocol格式协议的开源工具 [34]。

Autoprotocol基于JSON、人机可读的协议规范，是LLM实验室自动化函数调用模式的直接且高度相关的先驱。它对“维度值”和“refs/datarefs”的显式处理，为函数参数化和实验实体追踪提供了鲁棒模型，这对于LLM生成的协议能够执行和追溯至关重要。Autoprotocol是JSON序列化的 [35]，OpenAI使用JSON模式 [46]。Autoprotocol的“维度值”强制LLM生成精确、单位感知的参数（例如，“50:microliter”而不是简单的“50”），这对于实际实验室操作和安全至关重要，能防止歧义。其“refs/datarefs”功能使LLM能够管理和追踪实验输入（试剂、板）和输出（数据）的多个步骤，这对于多步骤协议生成和数据溯源至关重要。尽管它缺乏分支逻辑，但其简单性使其成为初期LLM训练顺序任务的明确目标。更复杂的逻辑可以通过外部编排或更高层次的LLM推理来处理。Autoprotocol的设计原则，源于对明确机器执行的需求，直接为创建既可被LLM解释又可被实验室执行的函数模式提供了最佳实践。因此，用户的数据集应明确使用JSON模式定义其函数模式和协议示例，并在适当情况下纳入Autoprotocol的语义元素（例如，“quantity”类型、ref/dataref模式）。这将提供一个强大且经过验证的基础。

#### 4.1.2 函数调用基准（例如，**Berkeley Function Calling Leaderboard**，**API-Bank**）

**Berkeley Function Calling Leaderboard (BFCL)**：一个综合性评估框架，用于评估LLM在各种语言、领域和复杂用例中的函数调用能力 [47]。它涵盖了多函数调用、并行函数调用、多轮函数调用和多步骤函数调用等任务 [47]。BFCL包含4951个测试用例（3951个单轮，1000

个多轮），侧重于动态的真实世界场景[47]。它在单轮（AST、可执行、实时）和多轮类别中进行评估，并评估幻觉现象[47]。

**API-Bank**：包含314个工具使用对话和753个API调用，用于评估LLM在规划、检索和调用API方面的能力，包括单次和多次调用[47]。APIGen是一个自动化数据生成管道，收集了21个类别中的3673个可执行API，以可伸缩和结构化的方式合成高质量的函数调用数据集，并通过格式检查、实际函数执行和语义验证进行验证[49]。

BFCL和API-Bank等基准的存在，评估LLM在复杂函数调用场景（多轮、并行、多步骤、幻觉）中的能力，为设计高质量的科学智能数据集提供了关键框架。这意味着数据集不仅应包含简单的函数调用，还应包含这些基准中发现的复杂性，以训练出用于真实实验室任务的鲁棒且可靠的LLM。BFCL和API-Bank评估多轮、并行、多步骤函数调用和幻觉[47]。APIGen通过执行和语义检查验证数据[49]。这些评估类别（单轮、多轮、并行、多步骤、幻觉）定义了科学智能LLM需要处理的交互类型。高质量的数据集应包含所有这些类别的示例。APIGen使用的数据验证过程（格式、执行、语义）强调了数据集中需要可执行且语义正确的函数调用，而不仅仅是语法正确的调用。对LLM函数调用能力进行严格评估的需求，推动了多样化和复杂基准的创建。这些基准反过来为构建涵盖真实世界复杂性和故障模式的全面训练数据集提供了路线图。因此，用户应设计其数据集，使其包含多轮对话（例如，LLM要求澄清，用户提供，LLM调用函数）、并行函数调用（例如，同时开始加热和搅拌）和多步骤协议。纳入负面示例或“幻觉”场景（LLM不应调用函数的情况）也将非常有价值。

#### 4.1.3 LLM生成数据与函数调用框架（例如，ChatLab，OpenBioset概念）

**ChatLab**：一个增强LLM计算能力的框架，允许用户编写Python函数并在Jupyter笔记本中与智能体交互[50]。它检查函数并为其生成JSON模式，捕获智能体响应和函数调用[50]。

**OpenBioset**（源自**OpenAI**函数调用用于数据生成）：OpenAI模型可用于自动化训练数据的创建，包括通过调用具有预定义模式的函数来标记文本或图像[51]。这确保了一致性，并允许逻辑处理数据（例如，保存到CSV）[51]。该过程涉及提供上下文、创建函数模式和定义函数[51]。

利用LLM自身生成函数调用训练数据（例如，OpenBioset概念）以及促进此过程的ChatLab等框架的出现，为数据集构建提供了一种可扩展且可能自我改进的方法。这意味着用户的数据集开发策略可以包括以人工策划的示例进行引导，然后通过LLM驱动的合成进行扩展，并由预定义模式和验证进行指导。LLM可以生成训练数据[51]。ChatLab从Python函数生成JSON模式[50]。[51]中描述的过程（提供上下文、创建模式、定义函数）本质上是生成合成函数调用数据的蓝图。如果用户定义目标实验室函数（其名称、描述、参数）并提供自然语言查询的示例，LLM可以生成更多查询-函数调用对。ChatLab自动从Python函数生成JSON模式的能力[50]简化了模式定义过程。对大规模、高质量函数调用数据集的需求推动了使用LLM进行自动化数据生成技术的发展。这使得数据集能够快速扩展，超越手动策展，从而加

速科学智能LLM的训练。因此，用户应探索混合数据生成策略：

- 人工策展：用于初始高质量、多样化的示例，特别是对于复杂或安全关键的场景。
- **LLM**合成：用于扩展数据集、生成变体和探索边缘情况，使用由预定义模式和验证规则指导的技术。这可能涉及一个迭代循环，其中人类专家审查和完善LLM生成的数据。

### 4.2 案例研究：LLM与自动化实验控制系统的集成

#### 4.2.1 自主合成规划（例如，IBM RXN）

概述：LLM正成为化学领域强大的工具，显著影响分子设计、性质预测和合成优化 [52]。基于LLM的自主智能体执行诸如论文抓取、与自动化实验室接口以及合成规划等任务 [52]。

集成：AI/ML和自动化的进步改变了化学合成优化，使得ML模型能够预测反应结果并以更少的实验找到最佳条件 [53]。闭环优化平台集成了实验室机器人，以最小的人工干预实现快速执行 [53]。工作流程包括实验设计、反应执行、数据收集、映射到目标以及预测下一步 [53]。

LLM的自主合成规划代表了一个完整的闭环科学智能应用，其中LLM不仅控制仪器，还积极参与科学方法（假设、实验、分析、迭代）。这意味着此类系统的函数调用需要支持复杂的、多模态的交互，包括数据解释和实验参数的迭代优化。LLM用于分子设计、性质预测和合成优化 [52]。闭环平台集成了机器人，以“最小的人工干预实现快速执行” [53]。合成规划涉及一系列化学反应，每个反应都有特定的条件和试剂。“闭环优化”意味着LLM需要解释结果（例如，产率、纯度），然后调整下一个实验的参数。这需要函数能够：a）定义复杂的反应步骤，b）指定精确的化学量和条件，c）触发分析测量，以及d）反馈结果以进行迭代规划。LLM的角色是高度认知性的，需要它对化学空间和实验结果进行推理。对更快、更高效的化学发现（业务/研究目标 ）的需求，推动了能够自主合成规划的AI智能体的开发。这要求函数调用接口能够处理化学反应的全部复杂性，并与分析反馈无缝集成以进行迭代优化。因此，数据集应包含多步骤化学合成协议的示例，其中函数允许指定反应物、溶剂、温度、反应时间、纯化步骤和分析表征。提示应反映复杂的化学目标。

#### 4.2.2 OpenAI函数调用在微型实验平台中的应用

概述：OpenAI函数调用使AI系统能够以结构化、可靠和可管理的方式与API交互 [5]。它赋予模型调用特定、由开发者定义的函数并带有详细参数的能力 [5]。

应用：模型识别提示何时与函数目的对齐，并可以建议或自动启动函数调用以检索数据、执行计算或与外部API交互 [5]。它能够将多个函数调用链接起来，实现复杂的多步骤过程 [5]。

设计原则：强调清晰的函数定义（名称、描述、参数、返回值类型）、明确的模式和精炼的提示 [5]。参数类型（枚举、整数、浮点数）对于模型理解预期输入至关重要 [55]。

OpenAI的通用函数调用框架为LLM如何与实验室设备接口提供了直接的架构模型。它对结

构化函数定义和参数JSON模式的强调[46]是用户数据集构建的关键技术细节。OpenAI函数调用使用“结构化函数定义”、“详细参数”和JSON模式[5]。OpenAI的JSON模式是LLM函数调用的事实标准。关于“函数模式的冗余性”、“参数类型”和“必需参数”的建议[55]是设计高质量实验室设备函数模式的直接、可操作指南。将多个函数调用“链接起来”的能力[5]对于多步骤实验室协议至关重要。对科学实验中高精度、安全性和可重现性的需求直接决定了函数模式设计和参数生成所需的严谨性。这意味着设计不佳的模式可能导致不安全或不可重现的实验，即使LLM生成了语法正确的调用。因此，用户的数据集应采用JSON模式格式来定义函数及其参数。每个实验室设备的功能应分解为离散、描述清晰的函数，并具有明确类型和必需的参数，遵循OpenAI的最佳实践。

#### 4.2.3 ChatGPT作为机器人实验室操作员：设计原则与应用

概述：研究探索了将ChatGPT应用于机器人领域的方法，通过结合提示工程和高层次函数库[56]。它能够适应不同的机器人任务、模拟器和外形尺寸[56]。

设计原则：关键概念包括创建高层次函数库[56]。API名称必须描述整体函数行为，以使LLM能够对函数连接进行推理[56]。它支持自由形式的自然语言对话、代码提示、XML标签和闭环推理[56]。

应用：证明了在基本逻辑、几何、数学推理以及空中导航和操作等复杂领域中的有效性[56]。目标是使用户主要通过自然语言进行交互[56]。

ChatGPT作为机器人实验室操作员的应用，特别是其对“高层次函数库”和“描述性API名称”的依赖，强化了函数调用中语义清晰度和层次抽象的必要性。这意味着数据集应优先选择代表有意义实验室操作的函数，而非原始机器命令，以促进LLM的推理。ChatGPT可以使用“高层次函数库”和“描述性API名称”作为机器人实验室操作员[56]。高层次函数库意味着像perform_cell_culture或synthesize_compound这样的函数比move_syringe_pump_to_position_X更有用。描述性API名称（例如，dispense_reagent而不是d_r_123）使LLM能够更好地理解函数目的并选择正确的工具。对“功能连接”进行推理的能力[56]对于复杂的、多步骤的协议至关重要。对与机器人进行直观自然语言交互的需求，推动了高层次、语义丰富的函数库的开发。这直接影响了函数调用数据集的设计，倾向于有意义、人类可解释的函数名称和描述。因此，在为数据集定义函数时，应优先使用清晰、简洁且描述性的名称和描述，准确反映所执行的科学操作。应尽可能避免过度技术化的行话或晦涩的缩写，以提高LLM将自然语言请求映射到正确函数的能力。

## 5. 未来趋势与科学智能函数调用数据集建设启发

### 5.1 推动实验自动化的新兴趋势

#### 5.1.1 从自然语言提示到结构化协议生成

LLM具备从科学文献中获取背景知识，从而促进协议设计的潜力[57]。目前正在开发框架，以

自动化协议从自然语言到结构化、可执行协议的翻译过程，这通常通过像协议依赖图（PDG）这样的中间表示来实现 [20]。提示工程是有效利用LLM的关键技能，涉及构建包含上下文、数据、任务和格式的提示 [58]。约束LLM（精确性、避免虚构、特定格式）和提供示例（一次性提示）是关键技术 [58]。LLM可以被微调，以从有机合成程序文本中提取反应信息，并按照如开放反应数据库（ORD）等模式将其结构化 [59]。

LLM生成从高层次自然语言提示到完整结构化实验协议的轨迹日益清晰。这意味着数据集构建应侧重于捕获完整的“提示到协议”映射，包括弥合语义鸿沟的中间推理步骤或结构化表示。LLM可以从自然语言设计协议 [57]。协议翻译框架存在 [20]。LLM可以从文本中提取结构化数据 [59]。从“提示到协议”的生成是一项复杂的任务，要求LLM理解科学意图，将其分解为步骤，选择适当的工具/函数，并正确参数化。使用“中间表示” [20] 和根据“开放反应数据库（ORD）模式”对“结构化数据”进行微调 [59] 表明LLM需要学习生成结构化输出，而不仅仅是自由形式的文本。提供上下文、示例和格式约束等提示工程技术 [58] 对于指导LLM至关重要。对自主实验设计的需求推动了LLM生成结构化、可执行协议的需求。这要求训练数据明确将自然语言提示与这些结构化表示联系起来，可能包括任务分解和参数推断的示例。因此，数据集应包含多样化的自然语言提示（例如，“合成化合物X”、“测量化合物Y在条件Z下的稳定性”），并与相应的结构化协议（函数调用序列，可能采用Autoprotocol或类似的JSON模式）配对。这将使LLM能够学习从高层次科学目标到具体实验室操作的映射。

### 5.1.2 LLM微调优化策略：模式设计、参数生成与接口控制

模式设计：对于LLM理解预期输入并生成准确响应至关重要 [55]。模式的冗余性会影响令牌数量和成本；参数应保持简单，非必要信息应隐藏 [55]。参数类型（整数、浮点数、枚举）优于通用字符串 [55]。必需参数应明确标记 [55]。

参数生成：LLM需要生成精确的参数。这通过清晰的函数定义和结构化的参数类型来支持 [46]。

接口控制：LLM可以增强计算能力，并在笔记本中与智能体交互（ChatLab [50]）。OpenAI函数调用能够将多个函数调用链接起来，实现复杂的多步骤过程 [5]。目标是为LLM提供一个“组织良好的工具包”，其中每个工具都清晰标记和描述 [5]。

LLM函数调用在科学智能中的成功取决于模式设计的精细程度，它需要在表达能力和简洁性之间取得平衡，从而实现精确的参数生成和鲁棒的接口控制。这意味着需要一个数据集，它不仅提供正确的函数调用，还展示如何有效地构建函数定义以及为何某些参数类型更适合科学精度和LLM性能。模式设计（冗余性、参数类型、必需字段）对于LLM的准确性和成本至关重要 [55]。LLM可以链式调用 [5]。科学实验要求高精度（例如，精确的体积、温度、持续时间）。资料中建议使用int、float、enum而非string [55]，这在该领域尤为关键，以防止LLM“幻觉”出不正确的数值或单位。“必需参数”的概念对于确保提供所有必要的安全或操作参数至关重要。“链式调用”能力 [5] 意味着LLM需要理解函数间的依赖关系，这影响了函数的分组方式或其输出如何成为后续调用的输入。对科学实验中高精度、安全性、可重现性的需

求直接决定了函数模式设计和参数生成所需的严谨性。这意味着设计不佳的模式可能导致不安全或不可重现的实验，即使LLM生成了语法正确的调用。因此，数据集应包含常见实验室操作的精心设计的函数模式示例，展示精确参数类型、单位和约束的使用。它还应包含多步骤协议的示例，其中参数生成依赖于中间结果或学习到的上下文，突出了接口控制的重要性。

### 5.2 构建高质量科学智能函数调用数据集的关键考量

#### 5.2.1 参考既定格式（例如，Autoprotocol，ChatGPT函数调用JSON模式）

**Autoprotocol**：基于JSON序列化的实验协议语言，定义了维度值、refs和datarefs [34]。它为顺序指令提供了清晰的结构。

**ChatGPT函数调用JSON模式**：使用JSON模式定义函数名称、描述和参数 [46]。强调清晰的参数类型（整数、字符串、枚举）和必需字段 [55]。

采纳或改编Autoprotocol和OpenAI的JSON模式等既定格式，是构建科学智能函数调用数据集的战略性要求。这确保了与现有工具和LLM训练范式的兼容性，加速了数据集开发和模型性能，同时为表示复杂的科学操作提供了经过验证的结构。Autoprotocol是基于JSON的 [35]，OpenAI使用JSON模式 [46]。使用既定格式减少了模式设计中“重新发明轮子”的需要。Autoprotocol为实验室协议提供了领域特定结构，而OpenAI的JSON模式为LLM函数调用提供了通用、广泛采用的标准。将它们结合（例如，在OpenAI风格的JSON模式中定义类似Autoprotocol的操作）可以兼顾两者的优点。显著偏离这些格式将需要定制的解析和训练，从而增加复杂性并降低互操作性。遵循既定格式可以减少开发开销并提高LLM兼容性，从而加速科学智能函数调用的进展。因此，数据集应明确使用JSON模式定义其函数模式，并在适当情况下纳入Autoprotocol的语义元素（例如，“quantity”类型、ref/dataref模式），以适应科学上下文。

#### 5.2.2 纳入关键元数据：安全性、延迟、设备状态与异步执行

安全性：实验室安全受多项法规约束（OSHA [60]）。AI驱动的系统必须考虑安全问题 [18]。函数调用可用于生成具有预定义模式的训练数据，确保一致性 [51]。

延迟与异步执行：“实时”数据处理和决策制定概念暗示了低延迟要求 [14]。PyLabRobot利用Python的asyncio实现并行执行 [8]。云实验室24/7运行，按需访问 [37]。

设备状态：PyLabRobot使用“跟踪器”（Tracker）对象来监测和验证机器人的物理状态（例如，吸头是否存在、液体状态） [8]。OSGi设备抽象层中的Device服务可以跟踪设备状态并读取描述性信息 [21]。

可模拟性：OSGi DAL中的Device服务可以表示纯软件单元，模拟真实设备的工作 [21]。PyLabRobot包含一个基于浏览器的模拟器 [8]。

将安全性、延迟、设备状态和异步执行模式等关键元数据纳入函数调用模式至关重要。这使得LLM能够生成更安全、更高效、更鲁棒的协议，能够应对真实世界实验室操作的复杂性和动态性。实验室环境固有的风险（例如，处理危险化学品、精密操作）要求LLM在生成指令时考虑安全性。这可能需要函数参数包含安全约束（例如，max_temperature、safe_volume_limit），或者元数据指示函数是否可以模拟。实时数据处理和高通量实验对低延迟和并行执行的需求，意味着函数调用模式应包含关于操作预期延迟和是否支持异步执行的信息。这有助于LLM优化协议执行顺序，避免瓶颈。LLM需要了解仪器的当前状态（例如，是否有吸头、是否有足够的试剂、是否正在运行）才能生成有效的后续命令。PyLabRobot的“跟踪器”对象和OSGi DAL的Device服务概念提供了如何将设备状态建模为函数调用的元数据。

这些元数据对于确保LLM生成的协议在实际实验室环境中是安全、高效和可执行的至关重要。缺乏这些信息可能导致不安全的操作、效率低下或无法执行的协议。因此，高质量的科学智能函数调用数据集应：

- 在函数模式中包含安全参数和约束：例如，temperature_range、volume_limit。
- 添加操作特性元数据：例如，expected_latency（预期延迟）、is_async_capable（是否支持异步）、requires_device_state_check（是否需要设备状态检查）。
- 提供可模拟性标记：指示函数是否可以在模拟环境中安全测试，这对于开发和调试至关重要。
- 包含设备状态信息：在多轮对话或复杂协议中，LLM需要访问设备当前状态的函数（例如，get_tip_count()、get_liquid_level()）。

## 6. 结论与建议

### 6.1 关键发现总结

本报告对科学智能中实验自动化和函数调用的现状进行了全面分析，揭示了以下关键发现：

1. 程序化控制的普及与接口多样性：无论是学术界还是工业界，液体处理系统、自动化显微镜和高通量筛选系统等核心实验室仪器普遍支持程序化控制。Python SDK、开放API（如Formulatrix）、LabVIEW以及REST API是常见的接口类型。其中，Python作为AI领域的主流语言，其SDK的普及极大地促进了LLM与实验室设备的集成。
2. 抽象层的重要性日益凸显：PyLabRobot等硬件无关的开源框架，以及OSGi设备抽象层和SiLA2等标准化协议，通过提供统一的命令集和设备行为表示，简化了LLM与异构仪器的交互。这种抽象层是实现LLM泛化能力和降低训练复杂度的关键。
3. **“自驱动实验室”**的闭环范式：伯克利、CMU和多伦多大学等高校，以及Insitro、Recursion等工业界领军企业，正在积极构建结合AI、机器人和实时数据反馈的闭环“自驱动实验室”。这些系统不仅执行实验，更能够自主设计、优化并迭代科学发现过程，对LLM提出了更高层次的智能体要求，即需要进行知识检索、实验设计和实时数据解释。

4. 商业云实验室的崛起：Strateos、Emerald Cloud Lab和Arctoris等商业云实验室平台，通过提供远程可访问的自动化基础设施和API，将实验室能力转化为“自动化即服务”。它们采用结构化协议语言（如Autoprotocol、SLL）来定义和执行实验，为LLM生成可执行协议提供了现实世界的参考。
5. 工业标准与**LLM**集成的潜力：SiLA2作为专为实验室自动化设计的开放标准，其“功能”、“命令”和“属性”的定义与函数调用模式高度契合，是LLM集成的理想蓝图。OPC UA的语义数据模型和ISA-88的批次控制“配方”概念，也为LLM理解和生成复杂实验流程提供了有益的结构化思路。
6. 函数调用数据集的结构化需求：现有基准（如BFCL、API-Bank）强调多轮、并行、多步骤和幻觉检测等复杂场景，表明高质量的科学智能数据集需要覆盖这些交互复杂性。Autoprotocol的JSON序列化协议和OpenAI的JSON Schema为函数模式设计提供了成熟的范例，特别是对维度值、实体引用和参数类型的精确定义。
7. **LLM**驱动数据生成的潜力：ChatLab等框架和利用LLM生成训练数据的概念，为大规模构建高质量函数调用数据集提供了可扩展的途径，有望加速AI4Science领域的发展。

### 6.2 数据集建设的行动建议

鉴于上述发现，为构建高质量的科学智能函数调用数据集，建议采取以下行动：

1. 优先采纳并扩展现有标准化的函数模式：
   - 以**SiLA2**和**Autoprotocol**为核心：深入研究SiLA2的“功能”（Features）、“命令”（Commands）和“属性”（Properties）模型，将其作为定义实验室仪器函数的基本框架。同时，借鉴Autoprotocol在协议JSON序列化、维度值（如“50:microliter”）和实体引用（refs/datarefs）方面的精确性，确保参数的科学严谨性与可追溯性。
   - 遵循**OpenAI**函数调用**JSON Schema**规范：采用其推荐的JSON Schema格式来定义函数及其参数，明确参数类型（例如，使用integer、float、enum而非泛型string），并清晰标记必需参数。这有助于提高LLM生成参数的准确性，并优化模型性能。
2. 构建多层次、多粒度的函数库：
   - 高层次抽象函数：包含代表复杂实验工作流的函数（例如，perform_cell_culture(cell_line, media_type, duration)、synthesize_compound(reactants, conditions)），这些函数在内部可分解为多个低层次操作。
   - 低层次原子操作函数：包含直接控制仪器基本动作的函数（例如，aspirate_volume(volume, source_well)、set_temperature(device, temperature)、capture_image(microscope, focus_plane)），确保LLM能够进行细粒度控制。
   - 数据管理与分析函数：纳入与LIMS/ELN系统交互的函数（例如，register_sample(sample_id, metadata)、log_experiment_data(run_id, results)、trigger_data_analysis(data_path)），以支持端到端的工作流程管理。

3. 集成关键元数据以增强鲁棒性与安全性：
    - 安全性元数据：为函数参数添加安全约束（例如，max_volume、max_pressure），并提供关于操作固有风险的元信息。考虑在函数描述中包含安全提示或警告。
    - 操作特性元数据：包含每个函数的预期延迟（expected_latency）、是否支持异步执行（is_async_capable）以及是否需要设备状态检查（requires_device_state_check）等信息。这有助于LLM在规划复杂协议时进行时间优化和错误处理。
    - 设备状态与可模拟性：设计函数以查询和更新设备状态（例如，get_tip_status()、check_reagent_level()）。对于可模拟的函数，添加指示符，以支持在虚拟环境中进行协议验证和调试。
4. 设计多样化且可验证的训练数据：
    - 覆盖复杂交互模式：数据集应包含单轮、多轮、并行和多步骤函数调用的示例，以训练LLM处理真实世界实验室任务的复杂性。
    - 包含“幻觉”场景：纳入LLM不应调用函数的负面示例，以提高模型的鲁棒性和安全性。
    - 强调可执行性与语义正确性：确保数据集中的函数调用不仅语法正确，而且在给定模拟或真实实验室环境的约束下是可执行且语义合理的。借鉴APIGen的验证流程（格式、执行、语义验证）。
5. 探索混合数据生成策略：
    - 人工策展与**LLM**合成结合：初期通过领域专家人工策展高质量、多样化的协议示例。随后，利用LLM（如通过ChatLab）根据预定义的函数模式和验证规则，大规模生成合成数据，以扩展数据集并探索边缘情况。建立人机协作的迭代循环，由人类专家审查和完善LLM生成的数据。

通过遵循这些建议，所构建的科学智能函数调用数据集将能够有效微调LLM，使其能够理解复杂的自然语言科研任务，并以安全、高效和可重现的方式，自动调用实验器材、控制实验流程或生成实验协议，从而加速科学发现的步伐。

**Works cited**


1. bio.tools · Bioinformatics Tools and Services Discovery Portal, accessed May 28, 2025, https://bio.tools/chemos
2. AI in Modern Biotech: Transforming Research & Innovations - Number Analytics, accessed May 28, 2025, https://www.numberanalytics.com/blog/ai-in-modern-biotech-transforming-research-innovations
3. AI4Science - van der Schaar Lab, accessed May 28, 2025, https://www.vanderschaar-lab.com/ai4science/
4. Function Calling: How AI Went from Chatbot to Do-It-All Intern : r/AI_Agents - Reddit, accessed May 28, 2025, https://www.reddit.com/r/AI_Agents/comments/1ivety9/function_calling_how_ai_

went_from_chatbot_to/
5. OpenAI Function Calling for API Integration and Automation - PromptLayer, accessed May 28, 2025, https://blog.promptlayer.com/openai-function-calling/
6. F.A.S.T. Liquid Handler - FORMULATRIX, accessed May 28, 2025, https://formulatrix.com/liquid-handling-systems/fast-liquid-handler/
7. FLO i8® PD Automated Liquid Handling System - Formulatrix, accessed May 28, 2025, https://formulatrix.com/liquid-handling-systems/flo-automated-liquid-handling/
8. PyLabRobot: An Open-Source, Hardware Agnostic Interface for ..., accessed May 28, 2025, https://pmc.ncbi.nlm.nih.gov/articles/PMC10369895/
9. PyLabRobot: An open-source, hardware-agnostic interface for liquid-handling robots and accessories — MIT Media Lab, accessed May 28, 2025, https://www.media.mit.edu/publications/pylabrobot-an-open-source-hardware-agnostic-interface-for-liquid-handling-robots-and-accessories/
10. Drug (API) Encapsulation Systems | Dolomite - Unchained Labs, accessed May 28, 2025, https://www.unchainedlabs.com/drug-api-encapsulation-systems/
11. Automated Microfluidic Instrument (AMI), accessed May 28, 2025, https://mems.utah.edu/research/ami/
12. Welcome to AEcroscoPy — AEcroscopy, accessed May 28, 2025, https://yongtaoliu.github.io/aecroscopy.pyae/welcome_intro.html
13. MunskyGroup/MicroscopyControlAndProcessing - GitHub, accessed May 28, 2025, https://github.com/MunskyGroup/MicroscopyControlAndProcessing
14. What role does AI play in high-throughput screening for drug discovery? - Patsnap Synapse, accessed May 28, 2025, https://synapse.patsnap.com/article/what-role-does-ai-play-in-high-throughput-screening-for-drug-discovery
15. High-Throughput Screening (HTS) by Cytometry Instruments - Sartorius, accessed May 28, 2025, https://www.sartorius.com/en/products/high-throughput-cytometry/high-throughput-cytometry-instruments
16. Self-driving lab - The Matter Lab, accessed May 28, 2025, https://www.matter.toronto.edu/basic-content-page/self-driving-lab
17. 'Self-driving labs': $200-million federal grant powers AI-driven materials discovery for clean energy, advanced manufacturing and more - U of T Engineering News - University of Toronto, accessed May 28, 2025, https://news.engineering.utoronto.ca/self-driving-labs-200-million-federal-grant-powers-ai-driven-materials-discovery-for-clean-energy-advanced-manufacturing-and-more/
18. CMU-Designed Artificially Intelligent Coscientist Automates Scientific Discovery - Mellon College of Science, accessed May 28, 2025, https://www.cmu.edu/mcs/news-events/2023/1220_ai-coscientist-automates-discovery.html
19. What's an AI Scientist? - The Automated Lab, accessed May 28, 2025, https://theautomatedlab.com/article.html?content=ai-scientist
20. Expert-level protocol translation for self-driving labs - NIPS papers, accessed May

28, 2025, https://proceedings.neurips.cc/paper_files/paper/2024/file/54dd9e0cff6d9214e20d97eb2a3bae49-Paper-Conference.pdf
21. 141 Device Abstraction Layer Specification - OSGi Compendium 8, accessed May 28, 2025, https://docs.osgi.org/specification/osgi.cmpn/8.1.0/service.dal.html
22. aspuru-guzik-group/ChemOS - GitHub, accessed May 28, 2025, https://github.com/aspuru-guzik-group/ChemOS
23. sorelle.friedler.net, accessed May 28, 2025, https://sorelle.friedler.net/papers/escalate_mrs19.pdf
24. Crystal structure prediction empowering solid-state chemistry solutions - XtalPi, accessed May 28, 2025, https://en.xtalpi.com/solid-state-chemistry-computation-solutions/
25. XtalGazer | Advanced Solid State Research for Drug Discovery - XtalPi, accessed May 28, 2025, https://en.xtalpi.com/xtalgazer/
26. Benchling API Integration - Teselagen, accessed May 28, 2025, https://teselagen.com/integrations/benchling-api-integration/
27. LIMS Automation & Integration | Benchling, accessed May 28, 2025, https://www.benchling.com/lims-benchling-provides-lims-automation-and-instrument-integration-capabilities
28. Technology - Recursion, accessed May 28, 2025, https://www.recursion.com/technology
29. Recursion: Pioneering TechBio Solutions in Drug Discovery, accessed May 28, 2025, https://www.recursion.com/
30. Assay Automation Software - Synthace Digital Experiment Platform, accessed May 28, 2025, https://www.synthace.com/applications/assays
31. Top 10 Tetrascience Alternatives & Competitors in 2025 | Trends - Scispot, accessed May 28, 2025, https://www.scispot.com/blog/top-tetrascience-alternatives-and-competitors
32. Robotic Controlled Life Sciences Lab - Strateos Cloud Lab, accessed May 28, 2025, https://strateos.com/
33. Strateos Lab Control Software - Automated Workflow Execution, accessed May 28, 2025, https://strateos.com/strateos-control-your-lab-version2/
34. Autoprotocol, accessed May 28, 2025, https://autoprotocol.org/
35. Specification - Autoprotocol, accessed May 28, 2025, https://autoprotocol.org/specification/
36. The Cloud Lab Revolution - 20/15 Visioneers, accessed May 28, 2025, https://www.20visioneers15.com/post/the-cloud-lab-revolution
37. Emerald Cloud Lab: Remote Controlled Life Sciences Lab, accessed May 28, 2025, https://www.emeraldcloudlab.com/
38. Cloud laboratory - Wikipedia, accessed May 28, 2025, https://en.wikipedia.org/wiki/Cloud_laboratory
39. The Lab of the Future Unveiled: How Technology is Transforming Scientific Discovery in 2025 | Trends - Scispot, accessed May 28, 2025, https://www.scispot.com/blog/the-lab-of-the-future-unveiled-how-technology-is-transforming-scientific-discovery

40. Industry experts offer OPC UA solutions update, accessed May 28, 2025, https://iebmedia.com/technology/opc-ua/industry-experts-offer-opc-ua-solutions-update/
41. SiLA Rapid Integration - Standards, accessed May 28, 2025, https://sila-standard.com/standards/
42. Enhancing reagent dispensing automation with SiLA 2 integration - News-Medical.net, accessed May 28, 2025, https://www.news-medical.net/whitepaper/20250310/Enhancing-reagent-dispensing-automation-with-SiLA-2-integration.aspx
43. Expanding automation into reagent dispensing with SiLA-2 integration - Integra Biosciences, accessed May 28, 2025, https://www.integra-biosciences.com/global/en/stories/expanding-automation-reagent-dispensing-sila-2-integration
44. ISA-88 Series of Standards - Batch Process Control, accessed May 28, 2025, https://www.isa.org/standards-and-publications/isa-standards/isa-88-standards
45. Introduction to Modbus - LabDeck, accessed May 28, 2025, https://labdeck.com/modbus/introduction-to-modbus/
46. Introspection for Function Calling - Amit Chaudhary, accessed May 28, 2025, https://amitness.com/posts/function-calling-schema/
47. ToolACE: Winning the Points of LLM Function Calling - OpenReview, accessed May 28, 2025, https://openreview.net/forum?id=8EB8k6DdCU
48. Tool Calling — Unitxt, accessed May 28, 2025, https://www.unitxt.ai/en/main/docs/tool_calling.html
49. NeurIPS Poster APIGen: Automated PIpeline for Generating Verifiable and Diverse Function-Calling Datasets, accessed May 28, 2025, https://neurips.cc/virtual/2024/poster/97756
50. Get Started - ChatLab, accessed May 28, 2025, https://chatlab.dev/docs/intro
51. Generating training data with OpenAI function calling - DEV Community, accessed May 28, 2025, https://dev.to/maurerkrisztian/generating-training-data-with-openai-function-calling-2c7l
52. A review of large language models and autonomous agents in chemistry - RSC Publishing, accessed May 28, 2025, https://pubs.rsc.org/en/content/articlehtml/2025/sc/d4sc03921a
53. Emerging trends in the optimization of organic synthesis through high-throughput tools and machine learning, accessed May 28, 2025, https://www.beilstein-journals.org/bjoc/articles/21/3
54. Helper functions to generate JSON schema dicts for OpenAI ChatGPT function calling requests. - GitHub, accessed May 28, 2025, https://github.com/jakecyr/openai-function-calling
55. Function calling with chat completion | Microsoft Learn, accessed May 28, 2025, https://learn.microsoft.com/en-us/semantic-kernel/concepts/ai-services/chat-completion/function-calling/
56. ChatGPT for Robotics: Design Principles and Model Abilities - Microsoft, accessed May 28, 2025,

https://www.microsoft.com/en-us/research/wp-content/uploads/2023/02/ChatGPT___Robotics.pdf
57. Hierarchically Encapsulated Representation for Protocol Design in Self-Driving Labs - arXiv, accessed May 28, 2025, https://arxiv.org/html/2504.03810v1
58. Best Practices for AI Prompt Engineering in Life Sciences - Certara, accessed May 28, 2025, https://www.certara.com/blog/best-practices-for-ai-prompt-engineering-in-life-sciences/
59. AI for Science: Current progress and pathways forward - ResearchGate, accessed May 28, 2025, https://www.researchgate.net/publication/389713839_AI_for_Science_Current_progress_and_pathways_forward
60. Laboratory Safety Guidance - OSHA, accessed May 28, 2025, https://www.osha.gov/sites/default/files/publications/OSHA3404laboratory-safety-guidance.pdf
61. Laboratory Equipment Safety | Environmental Health and Safety - University of Houston, accessed May 28, 2025, https://www.uh.edu/ehs/labs/general-laboratory-safety/equipment/index.php